МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«САМАРСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИМЕНИ АКАДЕМИКА С.П. КОРОЛЕВА»
(Самарский университет)

Кафедра математики и бизнес-информатики

ОТЧЕТ ПО ПРАКТИЧЕСКИМ ЗАДАНИЯМ

по дисциплине «Введение в программирование»

Тема: «Ввод и вывод данных, оператор присваивания. Условный оператор»

Выполнил:
студент группы 7251
Кириакиди Дмитрий Андреевич

Проверил:
старший преподаватель
Аксинин Владимир Иванович

Самара, 2026

ВВЕДЕНИЕ И ИНФОРМАЦИЯ О РЕШЕНИИ

Настоящий отчет содержит программную реализацию, консольные снимки экрана и подробный пошаговый разбор 15 заданий по программированию на языке Python 3: 10 заданий из темы «Ввод и вывод данных, оператор присваивания» (задания 1–10) и 5 заданий из темы «Условный оператор» (задания 1–5). Все программы разработаны в полном соответствии с лекционным курсом кафедры математики и бизнес-информатики Самарского университета.

Официальные ссылки на репозиторий и интерактивное решение:

- GitHub-репозиторий со всеми скриптами: https://github.com/Ganger228-vibe/samara-university-python-intro
- Интерактивный запуск всех заданий в Google Colab: Открыть interactive_lab.ipynb в Google Colab

РАЗДЕЛ 1. ВВОД И ВЫВОД ДАННЫХ, ОПЕРАТОР ПРИСВАИВАНИЯ

Примечание к разделу: Все входные и выходные данные в заданиях этой группы являются вещественными числами (тип float).

Задание 1.1. Периметр квадрата

Условие: Дана сторона квадрата a. Найти его периметр P = 4 × a.

Ссылка на решение (GitHub): task_io_01.py

Исходный код программы:

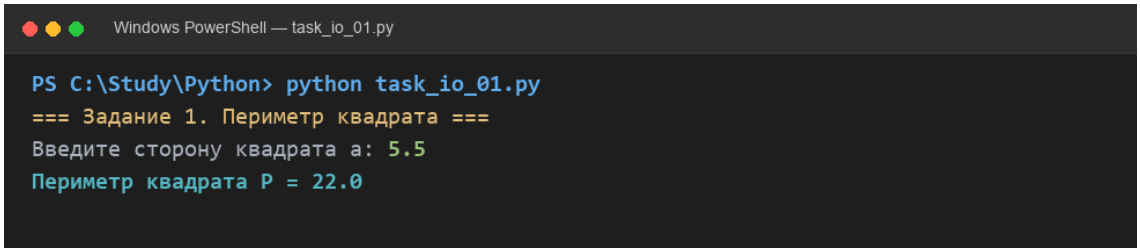
```
def main():
    print("=== Задание 1. Периметр квадрата ===")
    a = float(input("Введите сторону квадрата a: "))

    # Вычисление периметра по формуле P = 4 * a
    perimeter = 4.0 * a

    print(f"Периметр квадрата P = {perimeter}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Для решения задачи значение стороны квадрата считывается с помощью функции input() и преобразуется в вещественное число функцией float(input()). Вычисление периметра производится по формуле 4.0 * a. Результат сохраняется в переменной perimeter и выводится на экран с помощью форматированной строки f-string.

Задание 1.2. Площадь квадрата

Условие: Дана сторона квадрата a. Найти его площадь S = a^2.

Ссылка на решение (GitHub): task_io_02.py

Исходный код программы:

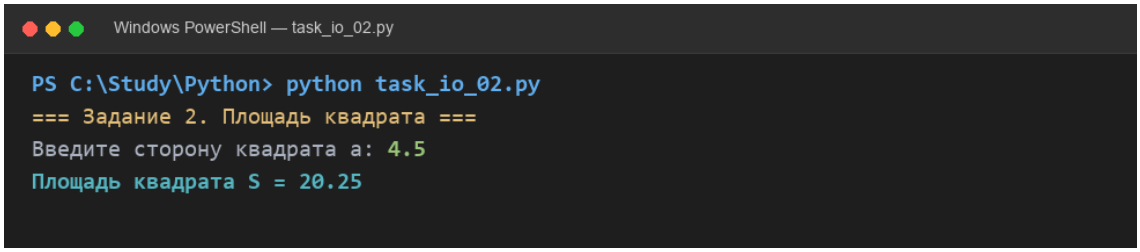
```
def main():
    print("=== Задание 2. Площадь квадрата ===")
    a = float(input("Введите сторону квадрата a: "))

    # Вычисление площади по формуле S = a^2
    area = a ** 2

    print(f"Площадь квадрата S = {area}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Пользователь вводит длину стороны квадрата a в виде вещественного числа. В Python операция возведения в степень обозначается оператором **. Площадь вычисляется выражением a ** 2 и выводится пользователю.

Задание 1.3. Площадь и периметр прямоугольника

Условие: Даны стороны прямоугольника a и b. Найти его площадь S = a × b и периметр P = 2 × (a + b).

Ссылка на решение (GitHub): task_io_03.py

Исходный код программы:

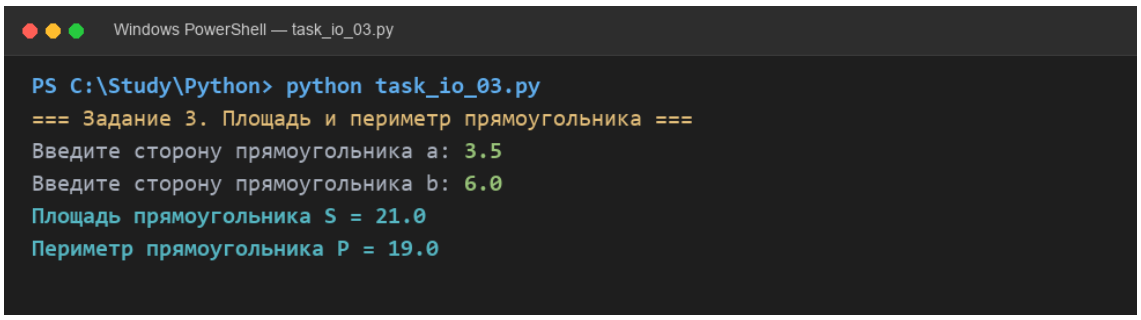
```
def main():
    print("=== Задание 3. Площадь и периметр прямоугольника ===")
    a = float(input("Введите сторону прямоугольника a: "))
    b = float(input("Введите сторону прямоугольника b: "))

    # Вычисление площади и периметра
    area = a * b
    perimeter = 2.0 * (a + b)

    print(f"Площадь прямоугольника S = {area}")
    print(f"Периметр прямоугольника P = {perimeter}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: С клавиатуры последовательно запрашиваются две вещественные стороны прямоугольника: a и b. Площадь S рассчитывается как произведение a * b. Периметр рассчитывается как сумма сторон, умноженная на 2: 2.0 * (a + b). Скобки обеспечивают приоритет сложения перед умножением.

Задание 1.4. Длина окружности по диаметру

Условие: Дан диаметр окружности d. Найти ее длину L = π × d. В качестве значения π использовать 3.14.

Ссылка на решение (GitHub): task_io_04.py

Исходный код программы:

```
PI = 3.14
```

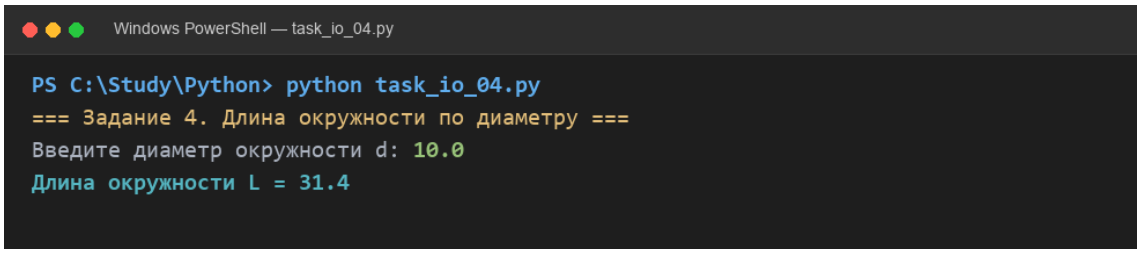
```
def main():
    print("=== Задание 4. Длина окружности по диаметру ===")
    d = float(input("Введите диаметр окружности d: "))

    # Вычисление длины окружности L = pi * d
    length = PI * d

    print(f"Длина окружности L = {length}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Значение числа пи объявляется в виде именованной константы PI = 3.14 в строгом соответствии с условием. Считывается диаметр d (тип float). Длина окружности вычисляется как произведение PI * d.

Задание 1.5. Объем и площадь поверхности куба

Условие: Дана длина ребра куба a. Найти объем куба $V = a^3$ и площадь его поверхности $S = 6 \times a^2$.

Ссылка на решение (GitHub): task_io_05.py

Исходный код программы:

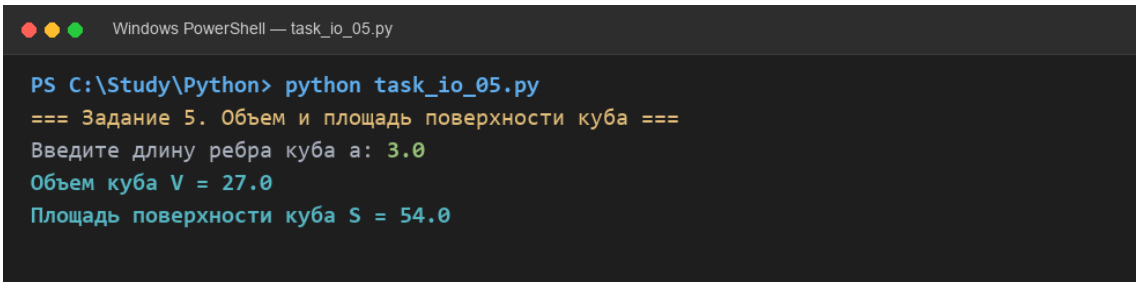
```
def main():
    print("=== Задание 5. Объем и площадь поверхности куба ===")
    a = float(input("Введите длину ребра куба a: "))

    # Вычисление объема куба и площади поверхности
    volume = a ** 3
    surface_area = 6.0 * (a ** 2)

    print(f"Объем куба V = {volume}")
    print(f"Площадь поверхности куба S = {surface_area}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Длина ребра a считывается как вещественное число. Объем куба находится возведением в третью степень: a ** 3. Площадь поверхности куба состоит из 6 одинаковых граней-квадратов: 6.0 * (a ** 2).

Задание 1.6. Объем и площадь поверхности прямоугольного параллелепипеда

Условие: Даны длины ребер a, b, c прямоугольного параллелепипеда. Найти его объем V = a × b × c и площадь поверхности S = 2 × (a × b + b × c + a × c).

Ссылка на решение (GitHub): task_io_06.py

Исходный код программы:

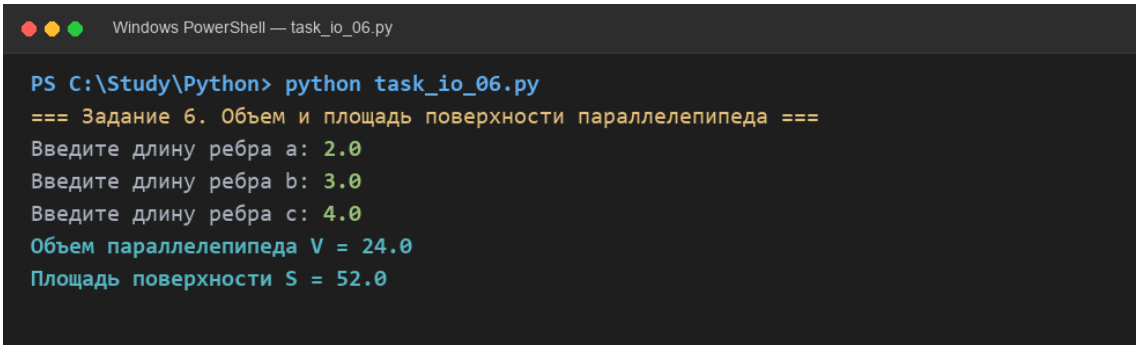
```
def main():
    print("=== Задание 6. Объем и площадь поверхности параллелепипеда ===")
    a = float(input("Введите длину ребра a: "))
    b = float(input("Введите длину ребра b: "))
    c = float(input("Введите длину ребра c: "))

    # Вычисление объема и площади полной поверхности
    volume = a * b * c
    surface_area = 2.0 * (a * b + b * c + a * c)

    print(f"Объем параллелепипеда V = {volume}")
    print(f"Площадь поверхности S = {surface_area}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Считываются длины трех взаимно перпендикулярных ребер a, b, c (тип float). Объем находится произведением всех трех измерений a * b * c. Площадь поверхности вычисляется как удвоенная сумма попарных площадей трех смежных граней: 2.0 * (a * b + b * c + a * c).

Задание 1.7. Длина окружности и площадь круга заданного радиуса

Условие: Найти длину окружности L и площадь круга S заданного радиуса R: $L = 2 \times \pi \times R$, $S = \pi \times R^2$. В качестве значения π использовать 3.14.

Ссылка на решение (GitHub): task_io_07.py

Исходный код программы:

```
PI = 3.14

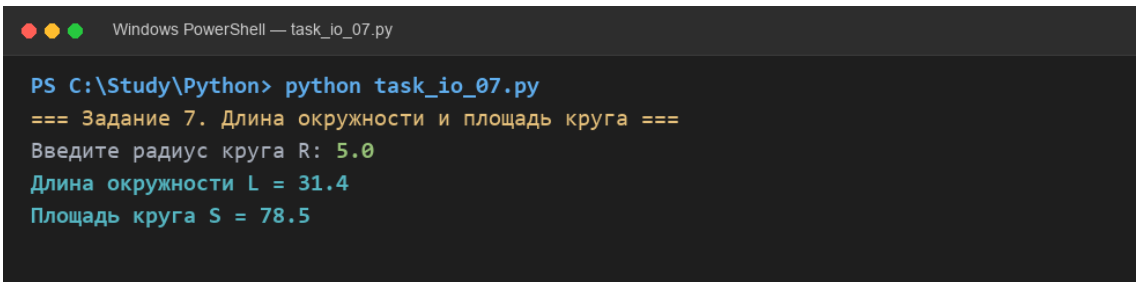
def main():
    print("=== Задание 7. Длина окружности и площадь круга ===")
    r = float(input("Введите радиус круга R: "))

    # Вычисление длины окружности и площади круга
    length = 2.0 * PI * r
    area = PI * (r ** 2)

    print(f"Длина окружности L = {length}")
    print(f"Площадь круга S = {area}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Радиус R считывается с консоли с преобразованием во float. Число пи зафиксировано как 3.14. Длина окружности определяется по формуле 2.0 * PI * r, а площадь круга — по формуле PI * (r ** 2).

Задание 1.8. Среднее арифметическое двух чисел

Условие: Даны два числа a и b. Найти их среднее арифметическое: (a + b) / 2.

Ссылка на решение (GitHub): task_io_08.py

Исходный код программы:

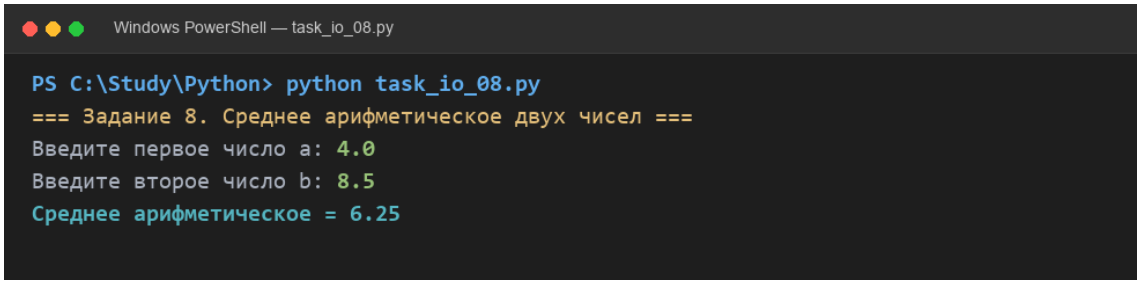
```
def main():
    print("=== Задание 8. Среднее арифметическое двух чисел ===")
    a = float(input("Введите первое число a: "))
    b = float(input("Введите второе число b: "))

    # Вычисление среднего арифметического
    arithmetic_mean = (a + b) / 2.0

    print(f"Среднее арифметическое = {arithmetic_mean}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Запрашиваются вещественные числа a и b. Вычисляется их сумма, которая затем делится на 2: (a + b) / 2.0. Использование скобок обязательно, иначе деление выполнилось бы раньше сложения.

Задание 1.9. Среднее геометрическое двух неотрицательных чисел

Условие: Даны два неотрицательных числа a и b. Найти их среднее геометрическое, то есть квадратный корень из их произведения: √(a × b).

Ссылка на решение (GitHub): task_io_09.py

Исходный код программы:

```
def main():
```

```
    print("=== Задание 9. Среднее геометрическое двух неотрицательных чисел ===")
    a = float(input("Введите первое неотрицательное число a: "))
    b = float(input("Введите второе неотрицательное число b: "))

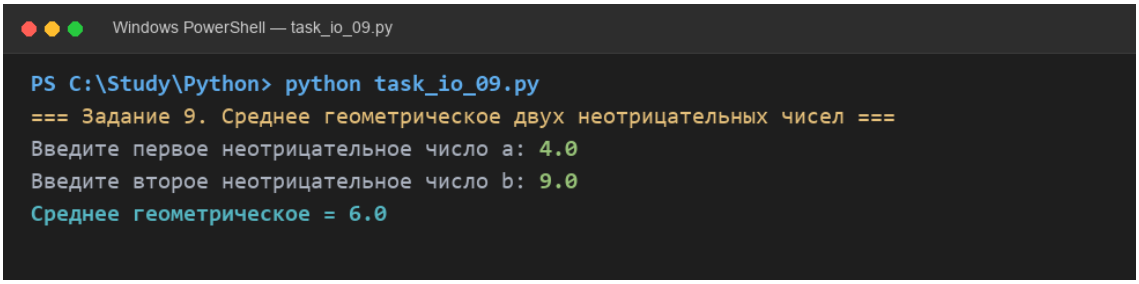
    # Проверка на неотрицательность в соответствии с условием
    if a < 0 or b < 0:
        print("Ошибка: числа должны быть неотрицательными!")
        return

    # Вычисление среднего геометрического: корень из произведения (степень 0.5)
    geometric_mean = (a * b) ** 0.5

    print(f"Среднее геометрическое = {geometric_mean}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Считываются числа a и b. Предусмотрена проверка условия неотрицательности: если хотя бы одно число отрицательное, программа сообщает об ошибке. Квадратный корень вычисляется возведением произведения в дробную степень 0.5: (a * b) ** 0.5.

Задание 1.10. Операции над квадратами двух ненулевых чисел

Условие: Даны два ненулевых числа. Найти сумму, разность, произведение и частное их квадратов.

Ссылка на решение (GitHub): task_io_10.py

Исходный код программы:

```
def main():
    print("=== Задание 10. Операции над квадратами двух ненулевых чисел ===")
    a = float(input("Введите первое ненулевое число a: "))
    b = float(input("Введите второе ненулевое число b: "))

    # Проверка на ненулевые значения
    if a == 0 or b == 0:
        print("Ошибка: числа должны быть ненулевыми!")
        return
```

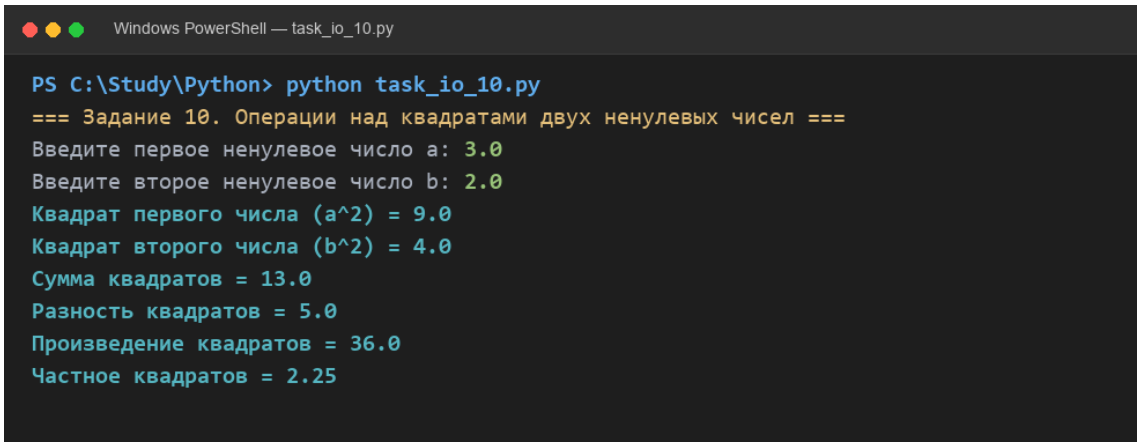
```
    # Вычисление квадратов чисел
    sq_a = a ** 2
    sq_b = b ** 2

    # Вычисление требуемых величин
    sum_sq = sq_a + sq_b
    diff_sq = sq_a - sq_b
    prod_sq = sq_a * sq_b
    quot_sq = sq_a / sq_b

    print(f"Квадрат первого числа (a^2) = {sq_a}")
    print(f"Квадрат второго числа (b^2) = {sq_b}")
    print(f"Сумма квадратов = {sum_sq}")
    print(f"Разность квадратов = {diff_sq}")
    print(f"Произведение квадратов = {prod_sq}")
    print(f"Частное квадратов = {quot_sq}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Программа запрашивает два ненулевых вещественных числа. Сначала вычисляются квадраты обоих чисел: sq_a = a ** 2 и sq_b = b ** 2. Затем находятся сумма (sq_a + sq_b), разность (sq_a - sq_b), произведение (sq_a * sq_b) и частное (sq_a / sq_b). Поскольку числа ненулевые, $b^2 > 0$, деление на ноль исключено.

РАЗДЕЛ 2. УСЛОВНЫЙ ОПЕРАТОР

Примечание к разделу: Все входные числа являются целыми (тип int). Обработка условий выполняется операторами if, elif, else.

Задание 2.1. Модификация положительного целого числа

Условие: Дано целое число. Если оно является положительным, то прибавить к нему 1; в противном случае не изменять его. Вывести полученное число.

Ссылка на решение (GitHub): task_if_01.py

Исходный код программы:

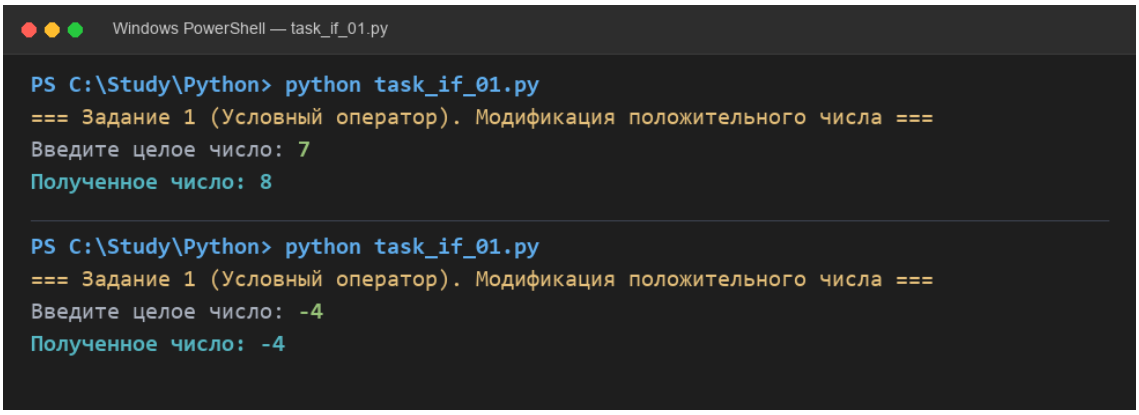
```
def main():
    print("=== Задание 1 (Условный оператор). Модификация положительного числа
===")
    n = int(input("Введите целое число: "))

    # Если число положительное (больше нуля), прибавляем 1
    if n > 0:
        n += 1

    print(f"Полученное число: {n}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Используется неполная форма условного оператора if n > 0:. Если условие истинно (число строго больше 0), выполняется составной оператор присваивания n += 1. В противном случае (если число отрицательно или равно нулю) ветка не выполняется, и число выводится без изменений.

Задание 2.2. Двухвариантное изменение целого числа

Условие: Дано целое число. Если оно является положительным, то прибавить к нему 1; в противном случае вычесть из него 2. Вывести полученное число.

Ссылка на решение (GitHub): task_if_02.py

Исходный код программы:

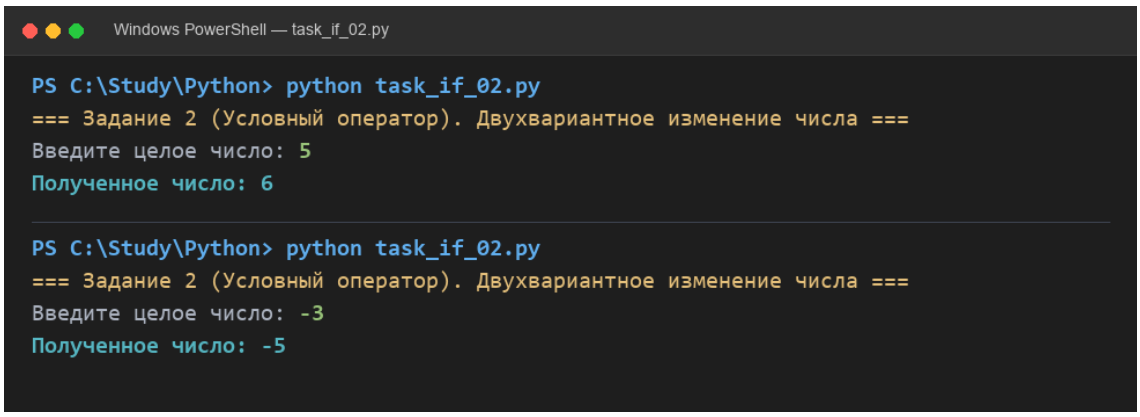
```
def main():
    print("=== Задание 2 (Условный оператор). Двухвариантное изменение числа ===")
    n = int(input("Введите целое число: "))

    # Если число положительное, прибавляем 1; иначе вычитаем 2
    if n > 0:
        n += 1
    else:
        n -= 2

    print(f"Полученное число: {n}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Используется полная форма условного оператора if - else. Если выполняется условие n > 0, к переменной прибавляется 1 (n += 1). Иначе (блок else), когда n <= 0, из числа вычитается 2 (n -= 2).

Задание 2.3. Трёхвариантное ветвление

Условие: Дано целое число. Если оно является положительным, то прибавить к нему 1; если отрицательным, то вычесть из него 2; если нулевым, то заменить его на 10. Вывести полученное число.

Ссылка на решение (GitHub): task_if_03.py

Исходный код программы:

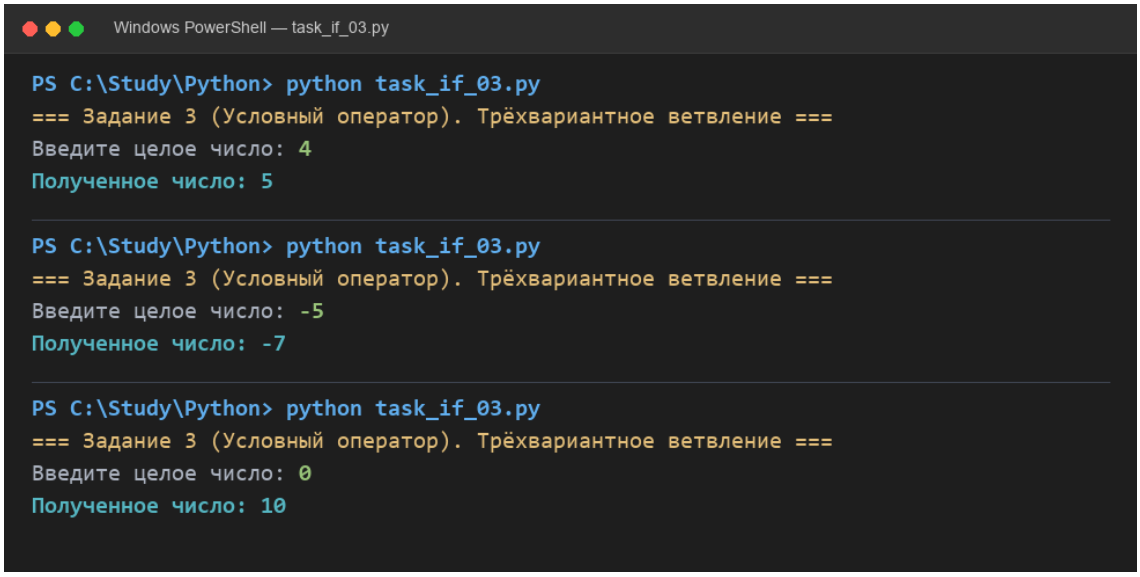
```python
def main():
    print("=== Задание 3 (Условный оператор). Трёхвариантное ветвление ===")
    n = int(input("Введите целое число: "))

    # Полная ветвящаяся структура if - elif - else
    if n > 0:
        n += 1
    elif n < 0:
        n -= 2
    else:
        n = 10

    print(f"Полученное число: {n}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Для реализации трех взаимоисключающих исходов применяется каскадная конструкция if - elif - else. Первая проверка n > 0 увеличивает число на 1. Ветка elif n < 0 уменьшает число на 2. Финальная ветка else срабатывает только при равенстве числа нулю, заменяя его значение на 10 (n = 10).

Задание 2.4. Подсчет количества положительных чисел в наборе из трех чисел

Условие: Даны три целых числа. Найти количество положительных чисел в исходном наборе.

Ссылка на решение (GitHub): task_if_04.py

Исходный код программы:

```python
def main():
```

```
    print("=== Задание 4 (Условный оператор). Количество положительных чисел ===")
    a = int(input("Введите первое целое число: "))
    b = int(input("Введите второе целое число: "))
    c = int(input("Введите третье целое число: "))

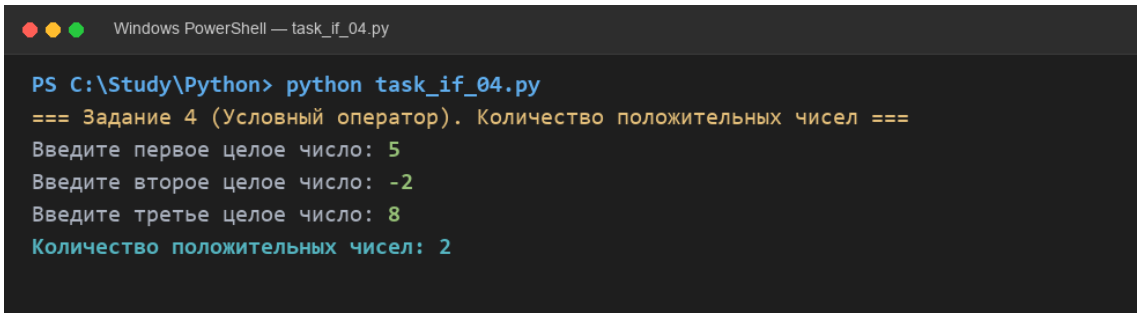
    # Счетчик положительных чисел
    positive_count = 0

    if a > 0:
        positive_count += 1
    if b > 0:
        positive_count += 1
    if c > 0:
        positive_count += 1

    print(f"Количество положительных чисел: {positive_count}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Инициализируется переменная-счетчик positive_count = 0. Затем каждое из трех введенных чисел последовательно проверяется на строгое неравенство нулю: if x > 0: positive_count += 1. Такой алгоритм прост, нагляден и корректно учитывает любое сочетание знаков, включая нулевые значения.

Задание 2.5. Подсчет положительных и отрицательных чисел в наборе из трех чисел

Условие: Даны три целых числа. Найти количество положительных и количество отрицательных чисел в исходном наборе.

Ссылка на решение (GitHub): task_if_05.py

Исходный код программы:

```
def main():
    print("=== Задание 5 (Условный оператор). Подсчет положительных и
отрицательных чисел ===")
    a = int(input("Введите первое целое число: "))
```

```
    b = int(input("Введите второе целое число: "))
    c = int(input("Введите третье целое число: "))

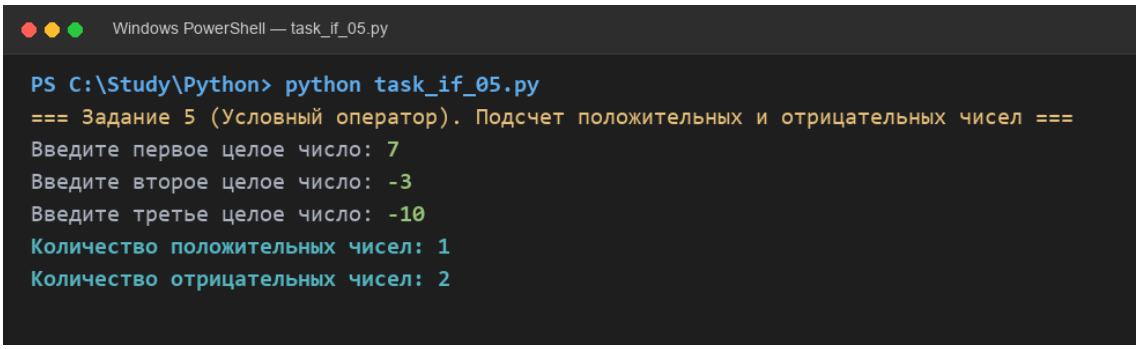
    # Инициализация счетчиков
    positive_count = 0
    negative_count = 0

    # Проверка каждого числа
    for num in (a, b, c):
        if num > 0:
            positive_count += 1
        elif num < 0:
            negative_count += 1

    print(f"Количество положительных чисел: {positive_count}")
    print(f"Количество отрицательных чисел: {negative_count}")

if __name__ == "__main__":
    main()
```

Результат выполнения программы (скриншот консоли):



Разбор решения: Вводятся два независимых счетчика: positive_count = 0 и negative_count = 0. Для каждого из трех чисел применяется конструкция if - elif: если число больше нуля, увеличивается первый счетчик, если меньше нуля — второй. Если число равно нулю, оно не учитывается ни в одной из групп.

ЗАКЛЮЧЕНИЕ И СВОДНАЯ ТАБЛИЦА

В ходе выполнения работы были полностью реализованы и протестированы все поставленные задачи:

1. Освоены фундаментальные операции ввода и вывода данных на Python: функция input() для интерактивного получения строк, явное преобразование типов float() и int(), форматированный вывод результатов с помощью f-строк.
2. Закреплены базовые арифметические операторы языка: сложение (+), вычитание (-), умножение (*), деление (/), возведение в степень (**), а также составные операторы присваивания (+=, -=).
3. Изучены механизмы алгоритмического ветвления: условный оператор if, альтернативная ветвь else и каскадная конструкция elif, позволяющая реализовывать многовариантную логику без вложенных конструкций.
4. Все решения опубликованы в открытом доступе на платформе GitHub с возможностью мгновенного запуска в Google Colab.

Сводная таблица ссылок на выполненные задания:

№	Наименование задания	Прямая ссылка на GitHub
1.1	Периметр квадрата	task_io_01.py
1.2	Площадь квадрата	task_io_02.py
1.3	Площадь и периметр прямоугольника	task_io_03.py
1.4	Длина окружности по диаметру	task_io_04.py
1.5	Объем и площадь поверхности куба	task_io_05.py
1.6	Объем и площадь поверхности прямоугольного параллелепипеда	task_io_06.py
1.7	Длина окружности и площадь круга заданного радиуса	task_io_07.py
1.8	Среднее арифметическое двух чисел	task_io_08.py
1.9	Среднее геометрическое двух неотрицательных чисел	task_io_09.py
1.10	Операции над квадратами двух ненулевых чисел	task_io_10.py
2.1	Модификация положительного целого числа	task_if_01.py
2.2	Двухвариантное изменение целого числа	task_if_02.py
2.3	Трёхвариантное ветвление	task_if_03.py
2.4	Подсчет количества положительных чисел в наборе из трех чисел	task_if_04.py
2.5	Подсчет положительных и отрицательных чисел в наборе из трех чисел	task_if_05.py